Quantum Bayesian Game Theory untuk Analisis Pasar

Teori dan Implementasi

February 5, 2026

Part I

Teori: Metodologi Quantum Bayesian Game Theory

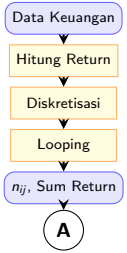
Pendahuluan

- Metodologi yang menjembatani dinamika pasar (**Econophysics**) dengan interaksi strategis (**Game Theory**)
- Menggunakan formalisme mekanika kuantum
- Menangkap "keterikatan" antar aset melalui probabilitas keadaan bersama
- Tidak sekadar korelasi linear, melainkan representasi **quantum state**

Tahap A: Persiapan Data

Langkah-langkah:

1. **Input Data**: Data Keuangan Leader (Gold) & Follower (Silver)
2. **Hitung Return**: Menggunakan `pct_change()`
3. **Diskretisasi State**: Return $< 0 \Rightarrow$ State 1 (Turun)
4. **Looping & Ekstraksi**: Menghasilkan n_{ij} dan Sum Return



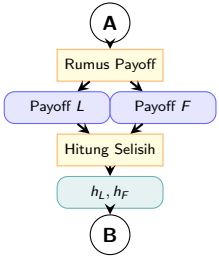
Output:

- Variabel n_{ij} (Frekuensi Kejadian)
- Variabel Sum Return (Total Return per State)

Tahap B: Matriks Payoff Game Theory

Langkah-langkah:

1. **Input**: Dari **(A)** n_{ij} dan Sum Return
2. **Rumus Payoff**: Payoff $= \frac{\text{Sum}}{n+1}$
3. **Hitung Selisih**: Payoff Naik $-$ Payoff Turun



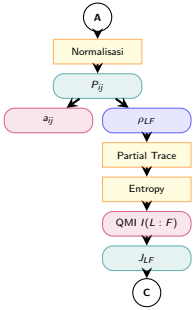
Output:

- Payoff Leader (a, c, e, g)
- Payoff Follower (b, d, f, h)
- Parameter Bias h_L, h_F

Tahap C: Probabilitas & Entanglement

Langkah-langkah:

1. **Input**: Dari **(A)** n_{ij}
2. **Normalisasi**: $P_{ij} = n_{ij}/n_{total}$
3. **Amplitudo**: $a_{ij} = \sqrt{P_{ij}}$
4. **Density Matrix**: $\rho = |\psi\rangle\langle\psi|$
5. **Partial Trace**: ρ_L, ρ_F
6. **Von Neumann Entropy**: $S = -\mathrm{Tr}(\rho \ln \rho)$
7. **QMI**: $I = S_L + S_F - S_{LF}$



Output: Parameter Interaksi J_{LF}

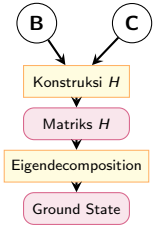
Tahap D: Model Hamiltonian

Langkah-langkah:

1. **Input**: Dari **(B)** h_L, h_F dan **(C)** J_{LF}
2. **Konstruksi Hamiltonian**:

$$H = -\sum h_i \sigma_i^z - J(\sigma_L^z \sigma_F^z)$$

3. **Eigendecomposition**: Mencari eigenvalues



Output:

- Matriks Hamiltonian H
- Ground State Energy
- Vektor Keadaan Paling Stabil

Skema Leader-Follower (Stackelberg)

- Pergerakan harga diperlakukan sebagai strategi
- Notasi bra-ket $|\cdot\rangle$ untuk merepresentasikan *state* diskrit (naik/turun)

Leader / Follower	$\|0\rangle_F$ (Up)	$\|1\rangle_F$ (Down)
$\|0\rangle_L$ (Up)	(a, b)	(c, d)
$\|1\rangle_L$ (Down)	(e, f)	(g, h)

Rumus Nilai Payoff

Nilai payoff dihitung menggunakan rata-rata return dengan Laplace smoothing:

$$a = \frac{\sum L^{00}}{n_{00} + 1}, \quad b = \frac{\sum F^{00}}{n_{00} + 1} \tag{1}$$

$$c = \frac{\sum L^{01}}{n_{01} + 1}, \quad d = \frac{\sum F^{01}}{n_{01} + 1} \tag{2}$$

$$e = \frac{\sum L^{10}}{n_{10} + 1}, \quad f = \frac{\sum F^{10}}{n_{10} + 1} \tag{3}$$

$$g = \frac{\sum L^{11}}{n_{11} + 1}, \quad h = \frac{\sum F^{11}}{n_{11} + 1} \tag{4}$$

Dimana n_{ij} adalah jumlah kejadian state (i, j)

Klasifikasi Tipe Permainan

Berdasarkan nilai payoff, hubungan antar aset dapat diklasifikasikan:

- **Coordination Game**: Payoff tertinggi pada $(0, 0)$ dan $(1, 1)$
  - Aset bergerak searah (korelasi positif kuat)
- **Anti-Coordination**: Payoff tertinggi pada $(0, 1)$ atau $(1, 0)$
  - Hubungan substitusi atau *hedging*
- **Prisoner's Dilemma**: Ada insentif "mengkhianati" arah tren

Nash Equilibrium: Jika $a > e$ dan $b > d$, maka $|00\rangle$ adalah equilibrium

Bayesian Game

- Matriks dapat menjadi **Bayesian Game**
- **Type**: Kondisi internal pasar (bullish/bearish tersembunyi)
- **Beliefs**: Probabilitas pergerakan lawan

Keadaan $|\psi\rangle$ merepresentasikan *Joint Probability Distribution* yang mendasari **Bayesian Nash Equilibrium**

Konstruksi Quantum State

Sistem dua aset direpresentasikan sebagai fungsi gelombang:

$$|\psi\rangle = a_{00}|00\rangle + a_{01}|01\rangle + a_{10}|10\rangle + a_{11}|11\rangle \tag{5}$$

- $|a_{ij}|^2 = P(|ij\rangle)$ adalah probabilitas gabungan
- Koefisien dihitung dari: $a_{ij} = \sqrt{P_{ij}}$

Reduced Density Matrix & Entanglement

Untuk mengukur derajat ketergantungan (entanglement):

1. Bentuk matriks densitas: $\rho = |\psi\rangle\langle\psi|$
2. Partial trace: $\rho_L = \mathrm{Tr}_F(\rho)$
3. Von Neumann Entropy:

$$S = -\mathrm{Tr}(\rho_L \ln \rho_L) \tag{6}$$

Interpretasi: Jika $S > 0$, kedua aset *ter-entangle* secara informasi

Quantum Mutual Information

$$I(L:F) = S(\rho_L) + S(\rho_F) - S(\rho_{LF}) \tag{7}$$

- Semakin tinggi nilai I, semakin kuat keterikatan
- Menunjukkan "kluster kuantum" di pasar keuangan
- Aset bergerak sebagai satu kesatuan fungsi gelombang

Quantum Financial Network

Untuk lebih dari 2 aset:

- **Nodes**: Setiap aset
- **Edges**: Nilai Entanglement Entropy atau Quantum Mutual Information

Metode ini memungkinkan:

- Melihat siapa yang berkorelasi dengan siapa
- Menemukan "penggerak utama" (Leader) menggunakan **Eigenvector Centrality**

Model Ising Hamiltonian

Hamiltonian merepresentasikan **total energi** atau **fungsi biaya** sistem:

$$H = -\sum h_i \sigma_i^z - \sum J_{ij} \sigma_i^z \sigma_j^z \tag{8}$$

Komponen:

- **Field Term**: $H_{bias} = -\sum_i h_i \sigma_i^z$
  - h_i dari selisih payoff naik vs turun
- **Interaction Term**: $H_{int} = -J_{LF}(\sigma_L^z \sigma_F^z)$
  - $J_{LF} > 0$: Coordination Game (Feromagnetik)
  - $J_{LF} < 0$: Anti-coordination

Pemetaan Parameter

Menentukan bias (h) dari payoff:

$$h_L = \frac{1}{2}\left[(a+c)-(e+g)\right], \quad h_F = \frac{1}{2}\left[(b+f)-(d+h)\right] \tag{9}$$

Menentukan interaksi (J) dari Quantum Mutual Information:

$$J_{LF} = \mathrm{sgn}(\rho_{LF}) \cdot I(L:F) \tag{10}$$

Dimana:

- $\mathrm{sgn}(\rho_{LF})$: Tanda korelasi antar aset ($+1$ jika positif, -1 jika negatif)
- $I(L:F)$: Quantum Mutual Information

Part II

Implementasi: Kode Python

Import Library

```python
import yfinance as yf
import numpy as np
import pandas as pd
import scipy.linalg as la
import networkx as nx
import matplotlib.pyplot as plt
```

Library yang digunakan:

- `yfinance`: Mengambil data historis aset
- `numpy`, `scipy`: Komputasi numerik dan aljabar linear
- `networkx`: Analisis jaringan

Fungsi Pengambilan Data

```python
def get_market_data(tickers, period="2y"):
    data = yf.download(tickers, period=period)['Close']
    returns = data.pct_change().dropna()
    return returns

# Eksekusi
tickers = ['GC=F', 'SI=F']  # Gold dan Silver
data_returns = get_market_data(tickers)
```

Fungsi Analisis Quantum Game

```python
def analyze_quantum_game(returns, leader_col, follower_col):
    # Tentukan state: 0 jika naik (up), 1 jika turun (down)
    L_state = (returns[leader_col] < 0).astype(int)
    F_state = (returns[follower_col] < 0).astype(int)

    # Inisialisasi frekuensi dan akumulasi return
    n_ij = np.zeros((2, 2))
    sum_L = np.zeros((2, 2))
    sum_F = np.zeros((2, 2))

    for i in range(len(returns)):
        l, f = L_state.iloc[i], F_state.iloc[i]
        n_ij[l, f] += 1
        sum_L[l, f] += returns[leader_col].iloc[i]
        sum_F[l, f] += returns[follower_col].iloc[i]
```

Perhitungan Payoff dan Amplitudo

```
    n_total = n_ij.sum()

    # Nilai payoff dengan Laplace Smoothing
    payoff_L = sum_L / (n_ij + 1)
    payoff_F = sum_F / (n_ij + 1)

    # Quantum State Coefficients (Amplitudo Probabilitas)
    prob_matrix = n_ij / n_total
    amplitudes = np.sqrt(prob_matrix)  # a00, a01, a10, a11

    return payoff_L, payoff_F, amplitudes, prob_matrix
```

Hasil Matriks Payoff

Output dari analisis Emas (GC=F) vs Perak (SI=F):

Matriks Payoff Leader (Gold):

$$\begin{bmatrix} 0.0104 & 0.0045 \\ -0.0044 & -0.0107 \end{bmatrix}$$

Matriks Payoff Follower (Silver):

$$\begin{bmatrix} 0.0204 & -0.0085 \\ 0.0071 & -0.0212 \end{bmatrix}$$

Quantum State Coefficients:

$$\begin{bmatrix} 0.684 & 0.357 \\ 0.334 & 0.541 \end{bmatrix}$$

Konstruksi Matriks Densitas

```python
# Matriks Densitas Global
rho_LF = probs / np.sum(probs)

# Reduced Density Matrix untuk Leader (trace out Follower)
rho_L = np.diag([probs[0,0] + probs[0,1], probs[1,0] + probs[1,1]])

# Reduced Density Matrix untuk Follower (trace out Leader)
rho_F = np.diag([probs[0,0] + probs[1,0], probs[0,1] + probs[1,1]])
```

Fungsi Von Neumann Entropy

```python
def von_neumann_entropy(rho):
    """Menghitung entropi S = -Tr(rho * log2(rho))"""
    eigvals = la.eigvalsh(rho)
    eigvals = eigvals[eigvals > 1e-12]
    return -np.sum(eigvals * np.log2(eigvals))

# Hitung Entropi
s_L = von_neumann_entropy(rho_L)
s_F = von_neumann_entropy(rho_F)
s_LF = von_neumann_entropy(rho_LF)

# Quantum Mutual Information
qmi = s_L + s_F - s_LF
```

Hasil Analisis Entropi

Metrik	**Nilai**
Entropi Emas (S_L)	0.9735
Entropi Perak (S_F)	0.9816
Quantum Mutual Information (I)	0.9724

Interpretasi: Entanglement sangat kuat. Emas dan Perak memiliki "jalur informasi" yang hampir identik.

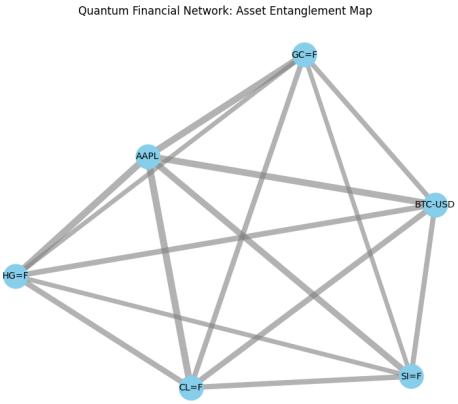
Membangun Quantum Financial Network

```python
# Definisikan daftar aset
tickers_list = ['GC=F', 'SI=F', 'CL=F', 'HG=F', 'BTC-USD', 'AAPL']

# Bangun Matriks Adjasensi berdasarkan QMI
G = nx.Graph()

for i, a in enumerate(tickers_list):
    for j, b in enumerate(tickers_list):
        if i < j:
            qmi_val = calculate_qmi_pair(all_data, a, b)
            G.add_edge(a, b, weight=qmi_val)
```

Visualisasi Quantum Financial Network



Komponen Graf:

- **Node**: Aset keuangan
- **Edge**: Kekuatan QMI antar aset

Interpretasi:

- Garis tebal $\Rightarrow$ entanglement kuat
- HG=F (Copper) memiliki banyak koneksi kuat
- GC=F & SI=F (Gold-Silver) terhubung erat
- BTC-USD relatif terpisah dari komoditas
- AAPL berperan sebagai "bridge" antar kluster

Eigenvector Centrality Analysis

```
# Cari Leader Utama dengan Eigenvector Centrality
centrality = nx.eigenvector_centrality(G, weight='weight')
sorted_centrality = sorted(centrality.items(),
                           key=lambda x: x[1], reverse=True)
```

Hasil Centrality Analysis (Market Leaders):

Asset	Centrality
AAPL	0.4642
CL=F	0.4158
BTC-USD	0.4054
HG=F	0.3980
GC=F	0.3812
SI=F	0.3789

Perhitungan Parameter Hamiltonian

```
# 1. Hitung parameter bias (h) dan interaksi (J)
h_L = (pL[0,0] + pL[0,1]) - (pL[1,0] + pL[1,1])
h_F = (pF[0,0] + pF[1,0]) - (pF[0,1] + pF[1,1])
J_LF = qmi  # Quantum Mutual Information sebagai kekuatan interaksi

# 2. Definisikan Operator Pauli-Z dan Identitas
Z = np.array([[1, 0], [0, -1]])
I_mat = np.eye(2)
```

Konstruksi Matriks Hamiltonian

```
# 3. Konstruksi Hamiltonian menggunakan Tensor Product
# H = -hL(Z x I) - hF(I x Z) - J(Z x Z)
H = -h_L * np.kron(Z, I_mat) - h_F * np.kron(I_mat, Z) \
    - J_LF * np.kron(Z, Z)

# 4. Mencari Ground State (Energi Terendah)
eigenvalues, eigenvectors = la.eigh(H)
ground_state_energy = eigenvalues[0]
ground_state_vector = eigenvectors[:, 0]
```

Hasil Matriks Hamiltonian

Matriks Hamiltonian Sistem (4x4):

$$H = \begin{bmatrix} -1.0596 & 0 & 0 & 0 \\ 0 & 0.9997 & 0 & 0 \\ 0 & 0 & 0.9451 & 0 \\ 0 & 0 & 0 & -0.8852 \end{bmatrix}$$

- **Energi Ground State**: -1.0596
- **Vektor Keadaan Paling Stabil**: $[1, 0, 0, 0]$ (keadaan $|00\rangle$)